HEIG VD | REDS Institut Reconfigurable and Embedded Digital Systems

Prof. Romuald Mosqueron

Prof. Olivier Auberson

Prof. Anthony Convers

Laboratoire de systèmes logiques
semestre automne 2026 - 2027

Laboratoire ADD/SUB, ALU et comparateur

Informations générales

Le rendu pour ce laboratoire se fera **par groupe de deux**. Chaque groupe devra rendre son travail avant la date mentionnée sur Cyberlearn.

Ce laboratoire se déroule sur **trois séances** et sera évalué de la façon suivante :

- Évaluation du circuit rendu
- Évaluation des réponses aux questions
- Validation de l'intégration sur la MAX_V Console **lors de la séance finale**

NOTE 1 : Afin de ne pas avoir de pénalité, pensez à respecter les points suivants :

- Toutes les entrées d'un composant doivent être connectées. (-0,1 sur la note par entrée non connectée)
- Lors de l'ouverture de Logisim, bien préciser vos noms en tant que User
- Ne pas modifier (enlever/ajouter/renommer/déplacer) les entrées/sorties déjà placées - sauf contre-indication explicite
- Merci de ne pas utiliser des portes XOR avec plus de 2 entrées.

NOTE 2 : Lors de la création de votre circuit, tenez compte des points suivants afin d'éviter des erreurs pendant la programmation de la carte FPGA :

- Nom d'un circuit $\neq$ Label d'un circuit
- Nom d'un signal (Pin) $\neq$ Label et/ou Nom d'un circuit, toutes les entrées/sorties doivent être nommées
- Les composants doivent avoir des labels différents

NOTE 3 : Nous vous rappelons que si vous utilisez les machines de laboratoire situées au niveau A, il ne faut pas considérer les données qui sont dessus comme sauvegardées. Si les machines ont un problème, nous les remettons dans leur état d'origine et toutes les données présentes sont effacées. Pensez à sauvegarder votre travail sur un autre support.

Outils

Pour ce laboratoire, vous devez utiliser les outils disponibles sur les machines de laboratoire (A07/A09) ou votre ordinateur personnel avec Logisim installé.

⚠ **La partie programmation d'un FPGA ne peut se faire que sur les ordinateurs présents dans les salles (A07/A09).**

Fichier Logisim fourni

Vous devez télécharger à partir du site Cyberlearn le projet Logisim dédié à ce laboratoire.

Le projet contient certaines des entités que vous allez réaliser dans le cadre de ce laboratoire. Vous devrez compléter ces entités et en créer de nouvelles afin de réaliser les fonctions demandées.

De plus, ne modifiez surtout pas les noms des entrées/sorties déjà placées dans ces entités et n'ajoutez pas d'entrée/sortie supplémentaires.

Conseil sur l'organisation du laboratoire

Le travail est réparti sur **trois séances** :

Séance	**Étapes à réaliser**
1	Réalisation de la partie arithmétique
2	Unité logique, Gestion des opcodes et sorties
3	Finalisation, implémentation et validation

TABLE 1 – Répartition du travail sur les séances

1 Contexte du laboratoire

L'objectif principal de ce laboratoire est la réalisation d'une unité arithmétique et logique 4 bits. Pour cela, vous devrez d'abord créer un certain nombre de composants puis les utiliser afin de concevoir l'ALU. L'idée est de développer un système de A à Z afin que vous puissiez faire chaque étape vous-mêmes et ainsi bien comprendre les concepts vus dans la théorie du cours afin de les appliquer dans un cas pratique.

Vous trouverez ci-après quelques définitions utiles concernant l'ALU à réaliser durant ce laboratoire.

1.1 Définition de l'ALU

Il s'agit du composant électronique essentiel au fonctionnement d'un CPU, d'un GPU ou d'un microcontrôleur, puisqu'il est chargé d'effectuer des opérations logiques et arithmétiques (soustraction, addition, multiplication...). Il s'agit en réalité du composant qui exécute les calculs.

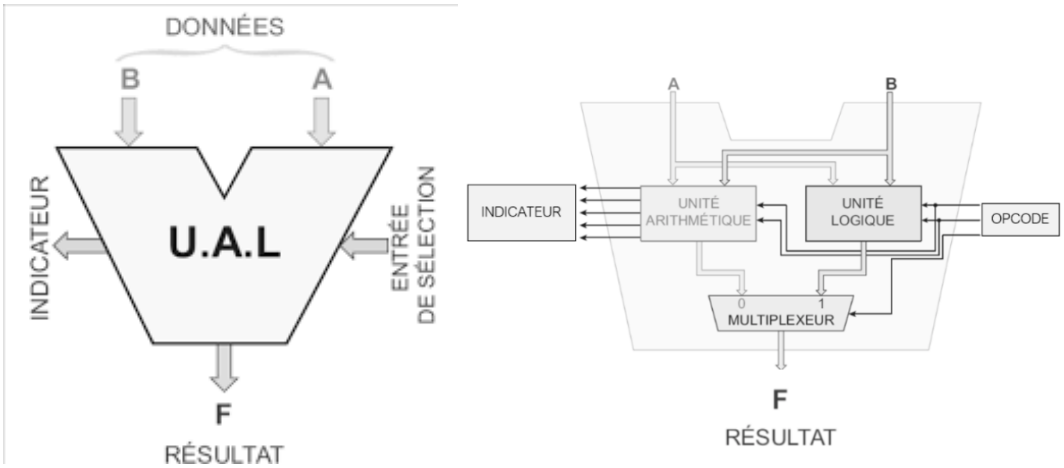


FIGURE 1 – Représentation de l'ALU et de son intérieur

Le composant prend en entrée deux valeurs A et B, effectue une opération choisie et en ressort un résultat F.

L'ALU renvoie également un indicateur. Cet indicateur permet de surveiller l'état du composant et les erreurs survenues : division par zéro, dépassement de capacité (overflow), opcode inconnu...

1.2 Les indicateurs NZCV

Dans un processeur, l'ALU met à disposition des indicateurs (aussi appelés flags). Ils forment un ensemble de bits permettant de statuer sur un résultat issu de l'ALU. Nous utiliserons les indicateurs NZCV. Ils indiquent si le résultat est **N**égatif ou égal à **Z**éro, ainsi que la valeur des bits de **C**arry et d'o**V**erflow.

1.3 Définition de l'Opcode

Pour connaître l'opération à effectuer, un code opération (ou opcode) est transmis à l'ALU. Ce code permet d'identifier l'unité à joindre et l'opération à effectuer au sein de ce composant.

Par exemple : si l'opération à effectuer est une soustraction, il faut joindre l'unité arithmétique et sélectionner l'opération de soustraction.

Les ALU sont présentes partout, mais peuvent avoir des fonctionnalités différentes selon l'utilisation qu'on souhaite leur donner *(exemple : calcul en virgule flottante (FPU)).*

1.4 Travail à effectuer

Dans le cadre de ce laboratoire, vous allez implémenter une ALU 4 bits capable de réaliser les opérations arithmétiques et logiques listées ci-dessous.

Opération	Fonctionnalité	Opcode `op_i[5:0]`
$A + B$ (signé)	Calculation	$0000\phi\phi$
$A - B$ (signé)		$0010\phi\phi$
$A + B$ (non signé)		$0001\phi\phi$
$A - B$ (non signé)		$0011\phi\phi$
$A \geq B$ (signé)	Comparateur	011001
$A < B$ (signé)		011010
$A \neq B$		011011
$A = B$		011100
$A \geq B$ (non signé)		011101
$A < B$ (non signé)		011110
A AND B	Logique	$10\phi\phi00$
A OR B		$10\phi\phi01$
A NAND B		$10\phi\phi10$
A XOR B		$10\phi\phi11$
Non défini		$11\phi\phi\phi\phi$

TABLE 2 – Table des opérations de l'ALU et leurs codes

1.5 Architecture de l'ALU

Cette ALU est composée de trois sous-blocs principaux (voir figure ci-dessous), chacun réalisant une fonctionnalité particulière. Ces trois blocs contiennent :

— **Bloc arithmétique** : réalise les opérations d'addition, de soustraction et de comparaison.
— **Bloc logique** : réalise les opérations logiques (AND, OR, NAND, XOR).
— **Bloc de gestion des sorties** : gère le résultat de l'opération choisie et l'indicateur d'erreur de l'ALU en fonction de l'opcode

La figure ci-après représente le schéma fonctionnel de l'ALU.

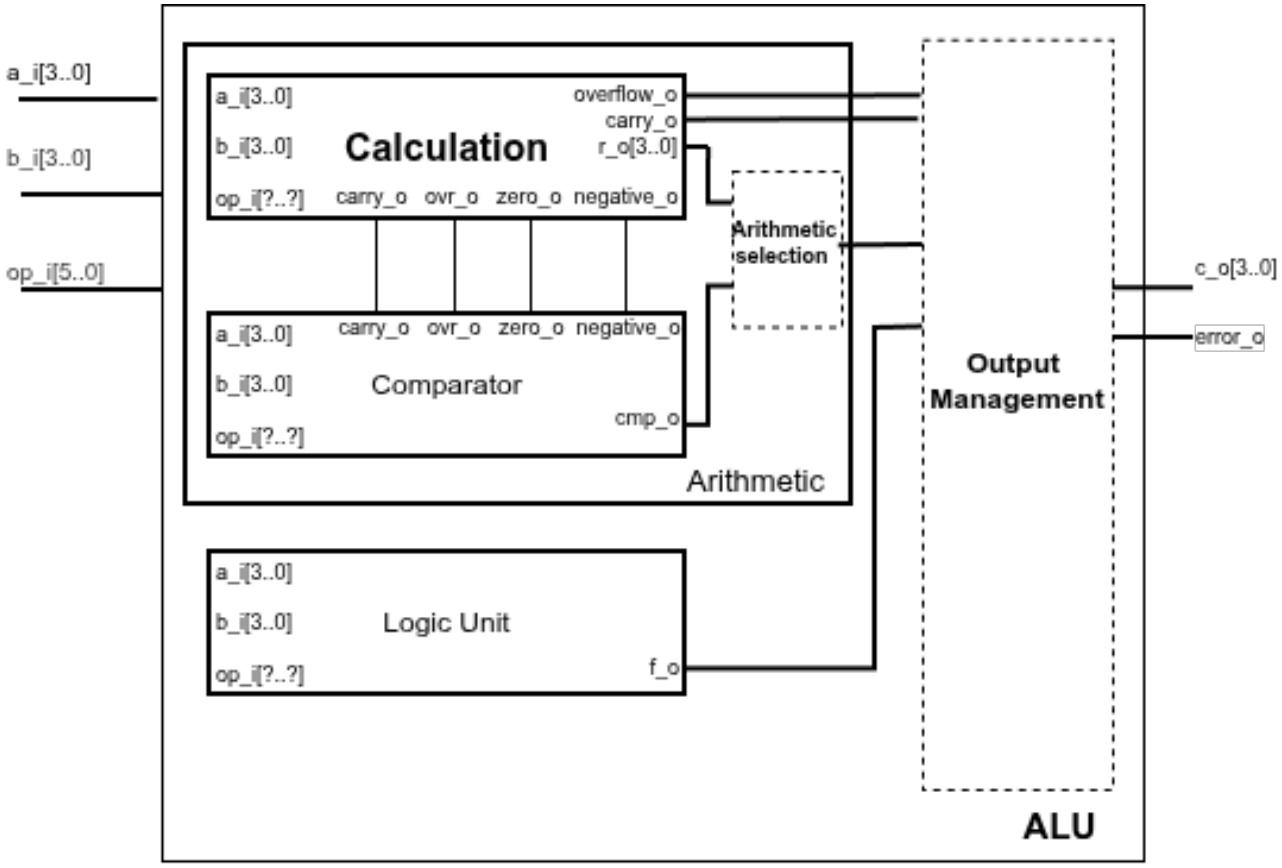


FIGURE 2 – Schéma fonctionnel de l'ALU

Avec comme entrées/sorties :

Entrée/Sortie	taille	Description
a_i	4	Premier opérande
b_i	4	Deuxième opérande
op_i	6	Opcode de l'opération
c_o	4	Résultat
error_o	1	Indicateur d'erreur

TABLE 3 – Entrées et sorties de l'ALU

2 Implémentation de la partie Arithmétique

2.1 Architecture du circuit de calcul

Voici le symbole et les entrées/sorties du système :

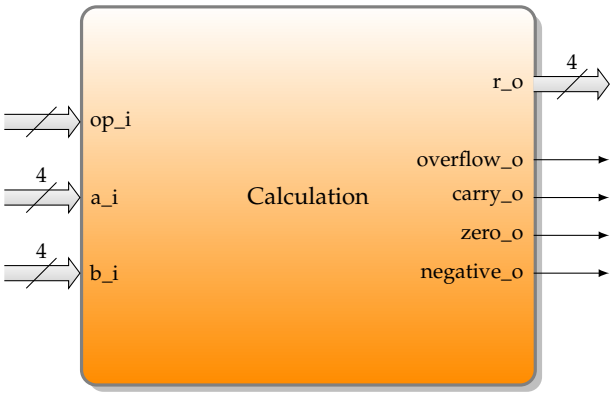


FIGURE 3 – Symbole et entrées/sorties du bloc Calculation

Avec les entrées et sorties suivantes :

Entrée/Sortie	taille	Description
op_i	?	Opcode de l'opération
a_i	4	Premier opérande
b_i	4	Deuxième opérande
r_o	4	Résultat
overflow_o	1	Indicateur de dépassement signé
carry_o	1	Indicateur de dépassement carry
zero_o	1	Indicateur de résultat nul
negative_o	1	Indicateur de résultat négatif

TABLE 4 – Entrées et sorties du bloc Calculation

2.2 Ajout de l'additioneur/soustracteur 4 bits

Pour cette partie, vous pouvez reprendre votre additionneur 4 bits développé lors du tutoriel de la première séance de laboratoire est implémenter l'add/sub tel que vu en cours.

2.3 Implémentation du bloc Calculation

À l'aide de votre additionneur/soustracteur 4 bits et des portes logiques, implémentez le circuit `Calculation` présenté à la page précédente.

Pour rappel, la table des opérations de l'ALU pour le bloc `Calculation` est donnée ci-dessous :

Opération	Opcode `op_i[5:0]`
$A + B$ (signé)	$0000\phi\phi$
$A - B$ (signé)	$0010\phi\phi$
$A + B$ (non signé)	$0001\phi\phi$
$A - B$ (non signé)	$0011\phi\phi$

TABLE 5 – Table des opérations de l'ALU pour le bloc `Calculation`

QUESTION 1 : Analysez et décrivez le comportement du bit de carry en sortie lors d'une opération de soustraction, en vous basant uniquement sur vos observations, sans apporter de modifications au circuit.

QUESTION 2 : Déduisez quelle est la taille minimale de l'entrée `op_i` du circuit `Calculation` pour réaliser toutes les opérations définies dans le tableau 5.

QUESTION 3 : En vous appuyant sur la table de vérité ci-dessous, obtenez l'équation simplifiée de la sortie overflow, puis intégrez la gestion de l'indicateur dans le composant `Calculation`.
⚠ Vous n'avez pas le droit d'utiliser la propriété vue en cours : $carry_n$ XOR $carry_{n-1}$ pour déterminer l'overflow.

Table de vérité vide de la sortie overflow :

Opération	A	B	R_o	overflow_o
$A + B$	≥ 0	≥ 0	≥ 0	
$A + B$	≥ 0	≥ 0	< 0	
$A + B$	≥ 0	< 0	≥ 0	
$A + B$	≥ 0	< 0	< 0	
$A + B$	< 0	≥ 0	≥ 0	
$A + B$	< 0	≥ 0	< 0	
$A + B$	< 0	< 0	≥ 0	
$A + B$	< 0	< 0	< 0	
$A - B$	≥ 0	≥ 0	≥ 0	
$A - B$	≥ 0	≥ 0	< 0	
$A - B$	≥ 0	< 0	≥ 0	
$A - B$	≥ 0	< 0	< 0	
$A - B$	< 0	≥ 0	≥ 0	
$A - B$	< 0	≥ 0	< 0	
$A - B$	< 0	< 0	≥ 0	
$A - B$	< 0	< 0	< 0	

TABLE 6 – Table de vérité de la sortie overflow pour le bloc `Calculation`

2.4 Implémentation du comparateur

La manière commune de réaliser une comparaison entre deux nombres A et B est d'effectuer l'opération $A - B$ et de se baser sur le résultat.

Le comparateur utilisera les indicateurs `NZCV` de l'ALU développée pendant la première séance.

Le symbole du comparateur est :

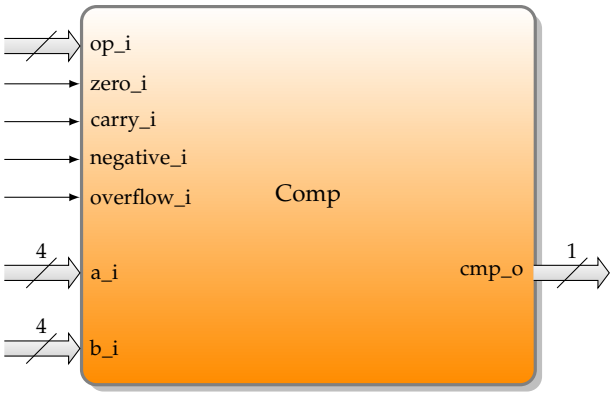


FIGURE 4 – Symbole du comparateur

Entrée/Sortie	taille	Description
op_i	?	Sélection de l'opération de comparaison
zero_i	1	Indicateur zéro
carry_i	1	Indicateur carry
negative_i	1	Indicateur négatif
overflow_i	1	Indicateur overflow
a_i	4	Premier opérande
b_i	4	Deuxième opérande
cmp_o	1	Résultat de la comparaison

TABLE 7 – Entrées et sorties du comparateur

Pour rappel, la table des opérations de comparaison de l'ALU est donnée ci-dessous :

Opération	Opcode `op_i[5:0]`
$A \geq B$ (signé)	011001
$A < B$ (signé)	011010
$A \neq B$	011011
$A = B$	011100
$A \geq B$ (non signé)	011101
$A < B$ (non signé)	011110

TABLE 8 – Table des opérations de comparaison de l'ALU

QUESTION 4 : De combien de bits l'opcode du comparateur a-t-il besoin au minimum pour pouvoir effectuer toutes ses opérations ?

Le comparateur possède les entrées `NZCV`, `a_i` et `b_i`. Il est possible de simplifier certaines comparaisons en n'utilisant pas certaines entrées lorsque cela est possible.

QUESTION 5 : Le comparateur tel qu'implémenté récupère plus d'informations que nécessaire pour certaines comparaisons. Identifiez quelles entrées peuvent être ignorées et expliquez pourquoi.

QUESTION 6 : Donnez les six équations de la sortie `cmp_o` pour chaque opération.

Implémentez le comparateur en utilisant les équations logiques obtenues à la question précédente.

⚠ Toutes les entrées de circuit non utilisées doivent être connectées à un symbole `Do not connect` sous `Wiring` dans Logisim.

2.5 Arithmetic Selection

Les blocs `Calculation` et `Comp` forment le bloc `Arithmetic` représenté dans le schéma fonctionnel de l'ALU.

Voici le symbole du bloc `Arithmetic Selection` :

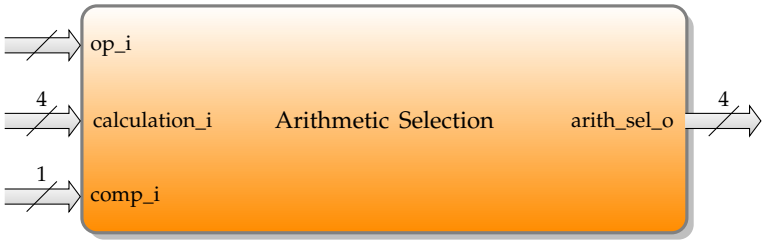


FIGURE 5 – Symbole du bloc Arithmetic Selection

Entrée/Sortie	taille	Description
op_i	?	Sélection de l'opération arithmétique
calculation_i	4	Sortie du bloc Calculation
comp_i	1	Sortie du bloc Comp
arith_sel_o	4	Résultat de la sélection arithmétique

TABLE 9 – Entrées et sorties du bloc Arithmetic Selection

Réalisez le bloc `Arithmetic Selection` en fonction des sorties de `Calculation` et `Comp`.

QUESTION 7 : De combien de bits l'opcode de sélection a-t-il besoin au minimum pour pouvoir sélectionner correctement les opérations arithmétiques ?

⚠ Le signal `cmp_o` étant défini sur 1 bit, il est assigné au bit de poids faible du résultat de l'ALU, c'est-à-dire `arith_sel_o`.

3 Implémentation de la partie Logique

3.1 Implémentation de l'unité logique

Réalisez le circuit `Logic`, en vous basant sur les spécifications définies dans le tableau 2.

Pour rappel, le symbole du bloc logique est :



FIGURE 6 – Symbole de l'unité logique

Entrée/Sortie	taille	Description
op_i	?	Sélection de l'opération logique
a_i	4	Premier opérande
b_i	4	Deuxième opérande
f_o	4	Résultat

TABLE 10 – Entrées et sorties de l'unité logique

Pour rappel, voici la table des opérations de l'unité logique de l'ALU :

Opération	Opcode `op_i[5:0]`
A AND B	$10\phi\phi00$
A OR B	$10\phi\phi01$
A NAND B (non signé)	$10\phi\phi10$
A XOR B (non signé)	$10\phi\phi11$

TABLE 11 – Table des opérations de l'unité logique

QUESTION 8 : De combien de bits l'opcode de l'unité logique a-t-il besoin au minimum pour pouvoir effectuer toutes ses opérations ?

3.2 Gestion des opcodes

Dans le circuit de l'`ALU`, déterminez comment chaque circuit interne utilise les opcodes pour correspondre aux différentes opérations. Ils doivent correspondre aux tailles minimales déterminées dans les questions 2, 4, 7 et 8.

3.3 Gestion des sorties

Implémentez la gestion des sorties de l'ALU en fonction des résultats de chaque circuit interne de l'`ALU` et des opcodes correspondants.

À vous de définir les entrées de ce circuit `OutputHandling`.

La sortie `error_o` doit indiquer si l'opération en cours génère une erreur ou si l'opcode ne correspond à aucune opération définie.

QUESTION 9 : Définissez le fonctionnement de la sortie `error_o` en fonction de chaque type d'opération (calculation, logique, comparaison).

4 Finalisation de l'ALU et intégration

4.1 Finalisation de l'ALU

Après avoir complété tous les circuits internes de l'ALU, intégrez-les dans le circuit principal de l'ALU et assurez-vous que toutes les opérations fonctionnent correctement.

4.2 Intégration sur la MAX_V Console

Les connexions pour l'intégration de l'ALU sur la MAX_V Console sont les suivantes :

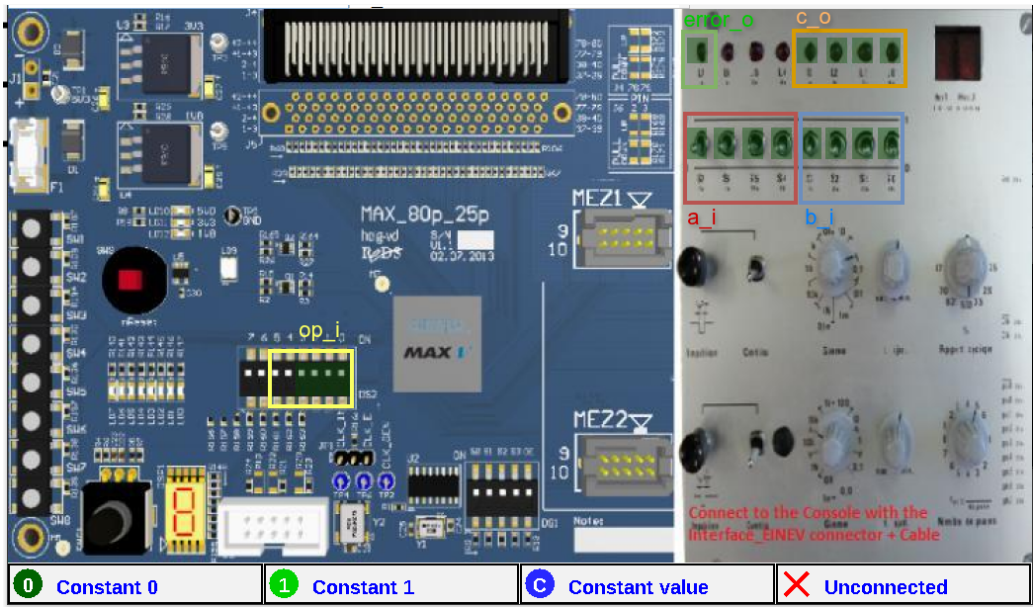


FIGURE 7 – Placement des entrées/sorties de l'ALU sur la MAX_V Console

4.3 Test et Validation

Après l'intégration de l'ALU sur la MAX_V Console, testez son fonctionnement pour vous assurer de son fonctionnement global.

La validation se fera :

1. Par l'**assistant** ou l'**enseignant** présent lors de la séance.
2. Vous avez **deux essais maximum** par binôme pour faire valider votre ALU.
3. La validation aura lieu uniquement lors de la dernière séance de ce laboratoire.

Rendu

Ce laboratoire est réalisé par groupe de deux.

Pour ce laboratoire, chaque groupe devra rendre une archive qui comporte les noms de votre binôme "rendu_NomEtudiant1_NomEtudiant2" et qui contient :

— Votre projet Logisim : **labo_alu_comp.circ**
— Un fichier **PDF** contenant les réponses aux différentes questions théoriques.

Vous devez déposer les rendus sur Cyberlearn jusqu'à la date indiquée dans l'espace de rendu consacré à votre classe.